Autonomous Internal Developer Platform (IDP): AI-Driven Infrastructure Orchestration and Local Container Provisioning

Domain Selection

- Primary Domain: Platform Engineering & Cloud-Native Infrastructure
- Sub-Domains: Agentic AI (Local LLM Orchestration) and Distributed Systems Management

Problem Statement

When software developers want to deploy their applications, they face two distinct bottlenecks:

- The "Ops" Tax: Developers waste hours writing complex Dockerfile configurations, managing ports, setting up reverse proxies, and dealing with environment inconsistencies instead of writing product logic.
- Data Privacy Leakage: Utilizing cloud-based AI tools (like public OpenAI or Claude APIs) to automate infrastructure generation exposes proprietary, enterprise source code to third-party servers, violating strict corporate data compliance.

The Solution: An entirely local, "Zero-Ops" Internal Developer Platform that ingests raw repositories, leverages a secure offline LLM to autonomously blueprint the required environment, and communicates directly with the system kernel to build and spin up isolated runtime containers instantly.

Abstract

Modern cloud-native architectures rely heavily on containerization and complex deployment pipelines, which frequently introduce significant operational overhead for software engineers. This project proposes a "Zero-Ops" Internal Developer Platform (IDP) designed to completely abstract away infrastructure management, enabling developers to deploy applications autonomously directly from raw source code.

The core architecture replaces traditional manual DevOps configurations with an Agentic AI workflow. By integrating a local, quantized Large Language Model (Llama 3.2 via Ollama), the platform programmatically analyzes repository codebases to deduce framework dependencies and autonomously generates optimal, secure Dockerfile configurations without exposing proprietary code to external cloud APIs.

The infrastructure orchestration layer is driven by a high-throughput backend (Node.js or Spring Boot) that bypasses standard CLI utilities to interface directly with the host's Docker Daemon via the Unix socket (/var/run/docker.sock). This direct integration enables the programmatic

execution of image compilation, dynamic container provisioning, and automated reverse-proxy port routing within an isolated environment.

To ensure strict operational transparency, the system establishes a bidirectional WebSocket connection to stream real-time compilation logs and container telemetry directly to a centralized React dashboard. Ultimately, this architecture demonstrates advanced Platform Engineering principles, effectively bridging the gap between zero-trust machine learning workflows and enterprise-grade distributed systems management.